

```

import itertools

def get_diagonal_cost(state):
    """Calculates the number of attacking queen pairs along diagonals."""
    cost = 0
    n = len(state)
    for i in range(n):
        for j in range(i + 1, n):
            if abs(state[i] - state[j]) == abs(i - j):
                cost += 1
    return cost

def get_permutation_neighbors(state):
    """Generates all 6 unique neighbors by swapping pairs of columns."""
    neighbors = []
    n = len(state)
    for col1, col2 in itertools.combinations(range(n), 2):
        neighbor = list(state)
        neighbor[col1], neighbor[col2] = neighbor[col2], neighbor[col1]
        neighbors.append(tuple(neighbor))
    return neighbors

def hill_climbing_4_queens(start_state):
    current = tuple(start_state)
    visited = set([current])

    print("\n" + "="*50)
    print(f"NAME: HRITHIKA RAJESH")
    print(f"USN : 1BM24CS120")
    print("="*50)
    print(f"--- STARTING STATE SPACE EXPLORATION ---")

    step = 1
    while True:
        current_cost = get_diagonal_cost(current)
        print(f"\n[Step {step}] Current State: {list(current)} (cost is {current_cost})")

        if current_cost == 0:
            print(f"🎉 GOAL STATE REACHED: {list(current)} with 0 conflicts!")
            break

        neighbors = get_permutation_neighbors(current)
        best_neighbor = None
        best_cost = current_cost

        print("Exploring all 6 permutation neighbors:")
        for neighbor in neighbors:
            cost = get_diagonal_cost(neighbor)
            is_visited = neighbor in visited
            status_flag = " (Visited/Repeated)" if is_visited else ""

            # Printed structure matches the textbook style perfectly
            print(f" * x0={neighbor[0]}, x1={neighbor[1]}, x2={neighbor[2]}, x3={neighbor[3]} | cost is {cost}{status_flag}")

            if not is_visited and cost < best_cost:
                best_cost = cost
                best_neighbor = neighbor

        if best_neighbor is not None:
            current = best_neighbor
            visited.add(current)
            print(f"-> Selected best unvisited neighbor: {list(current)} with cost {best_cost}")
            step += 1
        else:
            print(f"\n❌ Stuck in a local optimum or no better unvisited neighbor exists. Algorithm stops.")
            break

if __name__ == "__main__":
    print("--- 4-Queens State Space Configuration ---")
    try:
        # Allows you to provide a custom input state in comma-separated format
        user_input = input("Enter initial state values (comma-separated, e.g., 3,1,2,0): ")
        start = [int(x.strip()) for x in user_input.split(",")]

        if len(start) != 4 or set(start) != {0, 1, 2, 3}:

```

```

        print("⚠️ Error: Please enter a valid permutation of numbers 0, 1, 2, and 3.")
    else:
        hill_climbing_4_queens(start)
except ValueError:
    print("⚠️ Error: Invalid input format. Please use integers separated by commas.")

```

```

--- 4-Queens State Space Configuration ---
Enter initial state values (comma-separated, e.g., 3,1,2,0): 3,1,2,0

```

```

=====
NAME: HRITHIKA RAJESH
USN : 1BM24CS120
=====

```

```

--- STARTING STATE SPACE EXPLORATION ---

```

```

[Step 1] Current State: [3, 1, 2, 0] (cost is 2)
Exploring all 6 permutation neighbors:
* x0=1, x1=3, x2=2, x3=0 | cost is 1
* x0=2, x1=1, x2=3, x3=0 | cost is 1
* x0=0, x1=1, x2=2, x3=3 | cost is 6
* x0=3, x1=2, x2=1, x3=0 | cost is 6
* x0=3, x1=0, x2=2, x3=1 | cost is 1
* x0=3, x1=1, x2=0, x3=2 | cost is 1
-> Selected best unvisited neighbor: [1, 3, 2, 0] with cost 1

```

```

[Step 2] Current State: [1, 3, 2, 0] (cost is 1)
Exploring all 6 permutation neighbors:
* x0=3, x1=1, x2=2, x3=0 | cost is 2 (Visited/Repeated)
* x0=2, x1=3, x2=1, x3=0 | cost is 2
* x0=0, x1=3, x2=2, x3=1 | cost is 4
* x0=1, x1=2, x2=3, x3=0 | cost is 4
* x0=1, x1=0, x2=2, x3=3 | cost is 2
* x0=1, x1=3, x2=0, x3=2 | cost is 0
-> Selected best unvisited neighbor: [1, 3, 0, 2] with cost 0

```

```

[Step 3] Current State: [1, 3, 0, 2] (cost is 0)
🎉 GOAL STATE REACHED: [1, 3, 0, 2] with 0 conflicts!

```